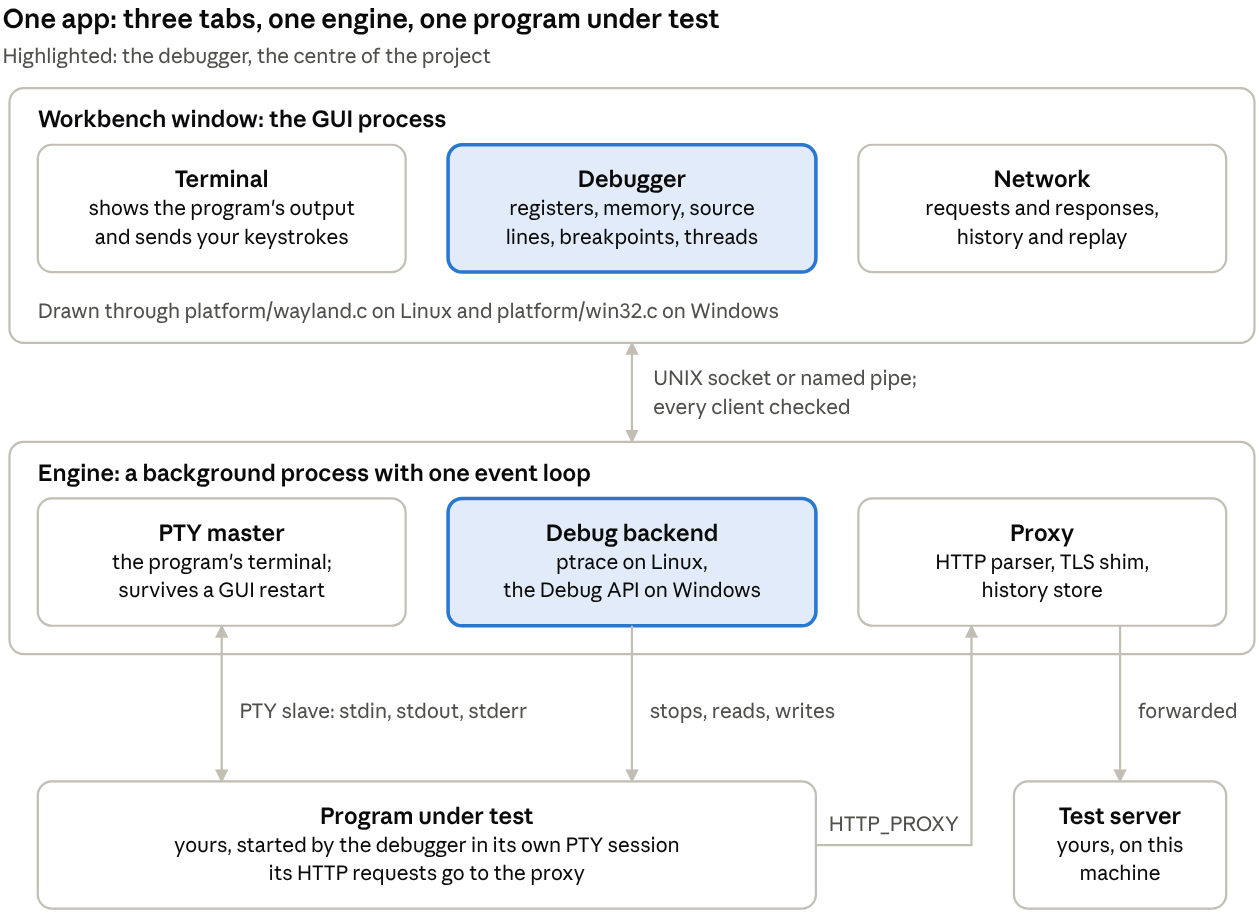
Workbench: Terminal, Debugger, Network Inspector

Nov 10, 2026 · @Murshid

Workbench is one custom-rendered C89 application for Linux and Windows, with three tools in one window: a terminal, a debugger for your own programs, and a network inspector. It replaces the Terminal Workbench plan after an external audit. The terminal stays, as the debugger's console. The debugger becomes the centre, because it forces every layer from the CPU up. The book chapters the product doesn't need become small labs instead of features.



Workbench architecture · 3 tabs, 1 engine, 1 program under test

The debugger and its backend are the centre. The terminal tab shows the program's own PTY, and the network tab sees the program's HTTP traffic through the proxy. The engine keeps all three alive when the window closes.

The decision

Build one application whose centre is a debugger for your own programs, with the terminal as its console and the network inspector as its third tab. A debugger is the only candidate that makes you handle, in working code, what the hardware and the operating system actually do: registers, the `int3` breakpoint instruction, the trap flag and debug registers, the ELF and PE loaders, memory maps, signal and exception delivery, threads and stack frames. That is "how my software actually runs", and it is one of your two targets.

What the audit changed

Terminal Workbench said	Workbench does
A terminal with panes added until most book chapters were touched	A debugger, with a terminal it needs anyway and a network tab you want anyway
Process, files, IPC-ring and plugin panes to cover TLPI chapters	Those chapters become **labs**: small standalone programs, each with a pass/fail test
A TCP mirror on localhost	Deleted: any local user can connect to 127.0.0.1, and TCP cannot tell who is connecting
"Every TLPI chapter from 4 to 64 except 38 gets code"	Honest tallies per book (see Coverage): substantial, but not that
Nothing on `ptrace`, ELF, DWARF, the Windows Debug API, Winsock or TLS	All of them, in the order your targets need them

What stays from Terminal Workbench

Its mental model, its graveyard and its hardware section are still correct and still apply: the PTY pair, the line discipline, sessions, `O_CLOEXEC`, draining SIGCHLD, reading in kernel-sized chunks, the 8-byte cell. Milestones 1–3 (window, terminal, shell) carry over almost unchanged. Read that document for the terminal side; this one covers the rest.

The repository

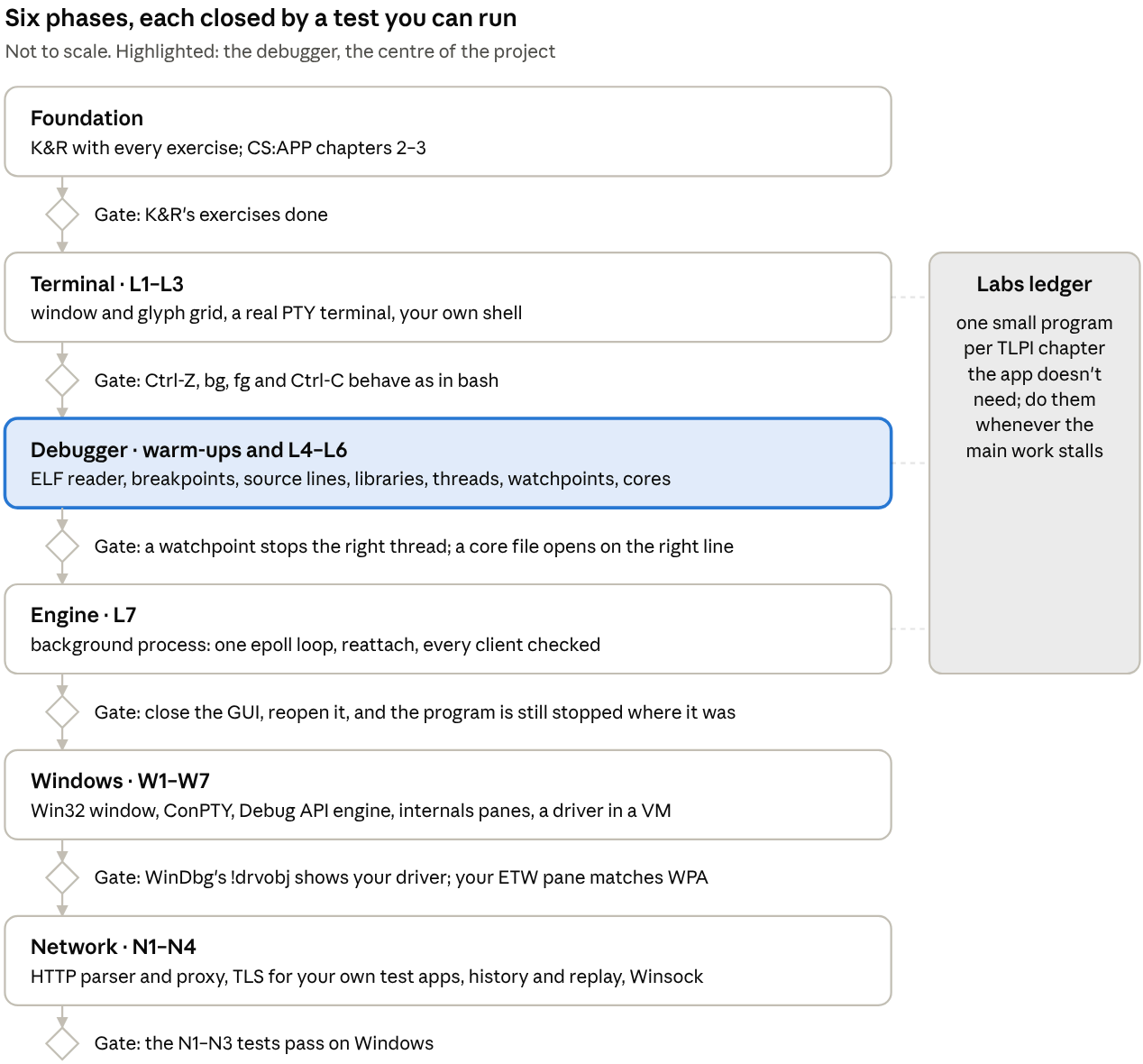
One repository, one portable core, one entry file per platform; labs and test programs live beside the app but are never linked into it.

```
workbench/
  core/          portable C89, no main(): grid, VT parser, renderer, arena,
                 ELF/DWARF readers, debugger engine, HTTP parser
  platform/      wayland.c, win32.c: the only files that talk to a windowing
system
  linux_main.c   the Linux app: #includes core/*.c and platform/wayland.c
  win32_main.c   the Windows app: WinMain, #includes core/*.c and
platform/win32.c
  labs/          one file per lab, each with its own main(): ch46_sysv_mq.c,
ch07_malloc.c, ...
  debuggees/     small test programs for the debugger to stop, step and
inspect
  build.sh       builds everything without Emacs: Linux, plus Windows via
MinGW
  .gitignore     .build/
  README.md
```

How it builds

- **Unity build per platform.** `linux_main.c` includes every `.c` file the Linux app needs, so the whole app is one translation unit and one compiler command. No makefile, no link order, and every function can be `static`.
- **Your Emacs block needs no change.** Tested on this exact layout: pressing `C-c C-c` in `core/vt.c`, `platform/wayland.c` or `linux_main.c` builds and runs the Linux app; in `labs/ch46_sysv_mq.c` or `debuggees/loop.c` it builds that one file alone. The `.git` folder stops the block's upward search at the repository.
- **`win32_main.c` uses `WinMain`, not `main`,** so on Linux the block never mistakes it for a second program.
- **`build.sh` is for the repository, not for you.** A repository must build without your editor: for the Windows build (MinGW cross-compiler), for anyone reading your code, and for automated builds later. The two Windows pieces that need Microsoft's compiler (the SEH labs and the driver) get their own build steps inside the VM.
- **`.build/` is ignored by git;** every output lands there.

Roadmap



Workbench roadmap · 6 phases, each with a gate

Each phase ends when its gate's test passes. The labs run beside the Linux phases, whenever the main work stalls.

Linux, part 1: window, terminal, shell

Three milestones give you a terminal you use every day, running a shell you wrote; the debugger later runs its programs inside it. Before L1, read Kernighan & Ritchie's *The C Programming Language* (2nd edition, which is C89) and do its exercises.

L1 — Window and glyph grid

- **Build:** the C89 core (cell grid, renderer, glyph atlas, arena) and `platform/wayland.c`, the only file that includes Wayland or xkbcommon headers. FreeType is called from the core. Port `wl_app.c` from the Third Document to C89 first: it was written for C17 (designated initializers, declarations mid-block, `//` comments).
- **You see:** a resizable window full of glyphs.
- **Done when:** a resize redraws at the new size without tearing; `grep -l 'wayland\|xkbcommon' core/*.c` prints nothing; `WAYLAND_DEBUG=1` shows one `wl_shm_pool` reused across frames.
- **Read before:** TLPI 4–5 (file I/O), 49 (mmap), 63 (poll); Drew DeVault, *The Wayland Protocol* (free online).
- **Keep open:** FreeType's tutorial; `WAYLAND_DEBUG=1`.

L2 — Real terminal

- **Build:** the PTY spawn from Terminal Workbench milestone 2 (already tested), plus an escape-sequence parser with the ground, escape and CSI states. Every child restores the signal mask before `exec`.
- **You see:** bash running inside the window.
- **Done when:** `ps -o pid,pgid,sid,tpgid,stat` shows the shell as `Ss` with PID = PGID = SID; `stty size` matches the window; `grep SigBlk /proc/<bash pid>/status` shows nothing inherited from your blocked set.
- **Read before:** TLPI 20–22 (signals), 24–27 (processes), 62 (terminals), 64 (pseudoterminals).
- **Keep open:** the DEC ANSI parser, XTerm's control-sequence list, `pty(7)`, `termios(3)`.

L3 — `c89sh`, plus the signal-handler lab

- **Build:** your own shell (pipelines, redirection, `fg`, `bg`, `jobs`, `limit`, `time`), plus one standalone lab in `labs/`: a SIGINT handler that only sets a `volatile sig_atomic_t`; a blocking `read` interrupted with and without `SA_RESTART`; a `sigaltstack` handler that reports a stack overflow. This lab is how TLPI chapter 21 gets coded even though the app itself uses no handlers.
- **You see:** a job-control shell as the terminal's default program.

- **Done when:** `sleep 100`, Ctrl-Z, `bg`, `fg` and Ctrl-C behave as in bash; the lab prints `EINTR` without `SA_RESTART` and not with it.
- **Read before:** TLPI 34 (job control), 36 (resources), 44 (pipes), 21 (handlers).

Linux, part 2: the debugger and its engine

The debugger launches the programs it debugs, inside the terminal you built, and every milestone is checked against gdb. It never attaches to a process it didn't start. That keeps you inside Linux's default tracing rule (a process may trace its own descendants) and inside "your own programs".

Warm-ups (before L4)

1. **Step a program by instruction.** `C-c m g` on your hello program, then `stepi` and `info registers`. Watch `rip` advance and `rsp` change across a `call`.
2. **Find `main` by hand.** `readelf -h`, `-S` and `-s` on `.build/hello.learn`; then follow `_start` in `objdump -d` until it calls `main`. `main` is not the first code that runs.
3. **Write a 100-line C89 ELF reader** that prints the header and the symbol table. It becomes `core/elf.c`, the debugger's symbol lookup.

L4 — Debugger core

- **Build:** start the program as a child that calls `ptrace(PTRACE_TRACEME)`, in its own PTY session with the terminal handed to it (`tcsetpgrp`). Software breakpoints: save the instruction's first byte, write `0xCC` (`int3`), and on a hit restore the byte, move `rip` back one byte, single-step, then re-insert. Single-step, registers, a memory-map pane from `/proc/PID/maps`, memory reads with `pread` on `/proc/PID/mem`.
- **Trap to expect:** your binaries are position-independent and load at a random base, so symbol addresses must be offset by the base in `/proc/PID/maps`. gdb disables address randomization for the programs it starts; do the same in the child, before `exec`, if you want your addresses to match gdb's.
- **You see:** a register pane and a hex memory pane over a stopped C89 test program.
- **Done when:** a breakpoint at `main` (address from your own ELF reader) hits; ten single-steps give the same `rip` sequence as gdb's `stepi`; your register values match `info registers`.
- **Read before:** CS:APP chapters 3 and 8; Sy Brand, *Building a Debugger*. **Keep open:** `ptrace(2)`, `elf(5)`, the x86-64 System V ABI.

L5 — Source lines and shared libraries

- **Build:** `mmap` the ELF file; parse only `.debug_line` (no types yet); a source pane; breakpoints on source lines; step over a line. Follow the dynamic loader's own breakpoint hook (`r_debug` and its `link_map` list) so you know when libraries load.
- **Done when:** a breakpoint on a source line inside a `dlopen`ed library fires after the library loads, and your module list matches the load order `LD_DEBUG=libs` prints.
- **Read before:** CS:APP chapter 7 (linking); TLPI 41–42. **Keep open:** the DWARF 5 line-number section; `readelf --debug-dump=decodedline`.

L6 — Threads, signals, watchpoints, core files

- **Build:** `PTRACE_O_TRACECLONE` and per-thread stops; passing signals through to the program; hardware watchpoints through the CPU's debug registers; opening core files.
- **Done when:** a write watchpoint on a global changed by the third thread stops with the right thread ID; a crashed test program's core file opens with the faulting instruction on the right source line.
- **Read before:** TLPI 29–33 (threads), 22 (core dumps). **Keep open:** Intel's manual, volume 3, the debug-registers chapter.

L7 — The background engine

- **Build:** the debugger engine becomes a background process (double fork, `syslog`, an `fcntl` lock for one instance) with one `epoll` loop over a UNIX socket, a `signalfd`, a `timerfd` and an `inotify` watch that reloads symbols when your binary is rebuilt. `SO_PEERCRED` checks every client. The program's PTY master goes to the GUI with `SCM_RIGHTS`; scrollback travels as a `memfd`.
- **You see:** close the GUI, reopen it, and the program is still stopped at the same instruction.
- **Done when:** reattach works; a second engine exits on the lock; a client of another user, connecting through a deliberately loosened test socket, is refused by *your* check and not just by folder permissions.
- **Read before:** TLPI 37 (daemons), 55 (locks), 56–57 and 61 (sockets), 63 (I/O models).

Labs ledger (L8)

Each TLPI chapter the product doesn't need gets one small program in `labs/`, time-boxed to a weekend or two, with a test that says pass or fail. Do them whenever the main work stalls; they don't block any milestone. Labs marked VM need root, so run them in a virtual machine.

Lab	TLPI	Build	Pass when
Your own allocator	7 (with CS:APP 9.9)	`my_malloc` / `my_free` over `sbrk` and `mmap`, with free lists	A million random-sized allocate/free cycles give the same checksums as glibc's `malloc`, with zero Valgrind errors
Buffering	13	Write 100 MB four ways: 1-byte `write`, stdio, 64 KiB `write`, and with `fsync` per block vs once	A table of four measured times you can explain
Attributes and ACLs	15–17	Read extended attributes and decode the raw POSIX ACL attribute	Your output matches `getfattr` and `getfacl` on files you set up
Libraries	41	A static and a shared library with a soname and an `rpath`	`ldd` shows your soname; bumping the major version breaks the old binary until relinked
System V IPC	45–48	Producer and consumer processes over a message queue, a semaphore set and a shared segment	Data arrives intact; `ipcs` shows nothing left after cleanup
POSIX IPC	51–54	The same over `mq_open`, `sem_open` and `shm_open`	Data arrives intact; `/dev/mqueue` and `/dev/shm` are empty after cleanup

Lab	TLPI	Build	Pass when
Memory operations	50	A guard page with `mprotect`; a residency map with `mincore`	Touching the guard page raises SIGSEGV at exactly its address; the residency map fills when you read the file and empties after `posix_fadvise(POSIX_FADV_DONTNEED)`
Login records	40	A read-only utmp reader with `getutxent`	Output matches `who`
Mounts (VM)	14	Mount a tmpfs and a bind mount	Your `/proc/self/mountinfo` reader shows both, with `statvfs` sizes matching `df`
Set-user-ID (VM)	9, 38	A set-user-ID program that drops privilege permanently	After the drop, `getresuid` shows the real user three times and `setuid(0)` fails
Capabilities (VM)	39	`setcap cap_net_bind_service` on a test server	It binds port 80 as an ordinary user; removing the capability makes `bind` fail with `EACCES`

The signal-handler lab for chapter 21 lives with L3, because the shell is where you meet that chapter's problem.

Windows: the same app, a second platform layer

The core is unchanged; `platform/win32.c` and `win32_main.c` replace Wayland, and the debugger engine gains a Windows backend. Your DWARF line reader carries over: MinGW builds put DWARF debug info inside the PE file (tested here: a MinGW `-g` build has `.debug_line` and `.debug_info`, and `objdump` decodes its line table).

W1 — Win32 window, DIB section, GDI glyphs

- **Build:** your own window procedure; UTF-16 throughout; `WM_CHAR` / `WM_KEYDOWN`; `WM_TIMER` for the blink; glyphs drawn once with `ExtTextOut` into a memory DC to fill the atlas; scroll bars. Drawing glyphs through GDI instead of FreeType is what turns Petzold's text, drawing and font chapters (4, 5, 17) into real code.
- **Done when:** the same escape-sequence test file renders like Linux, fonts aside; the core compiles with MinGW `-std=c89 -pedantic-errors`.

W2 — ConPTY terminal, resources, clipboard, printing

- **Build:** `CreatePseudoConsole` with two pipes; a menu bar and accelerators from a resource script; a settings dialog with child controls; copy and paste of scrollback; printing a session.
- **Done when:** PowerShell runs inside the window; menus and dialogs work from the keyboard alone; scrollback copies to the clipboard and prints.

W3 — Windows debugger engine

- **Build:** `CreateProcess` with `DEBUG_ONLY_THIS_PROCESS`; a `WaitForDebugEvent` / `ContinueDebugEvent` loop; `int3` through `WriteProcessMemory` plus `FlushInstructionCache`; single-step through the trap flag with `GetThreadContext` / `SetThreadContext`; DLL-load and thread-create events; first- and second-chance exceptions. A PE reader replaces the ELF reader; the DWARF reader is reused.
- **Done when:** a breakpoint on `main` of a MinGW-built test program hits, and a first-chance access violation is reported before the program's own `__except` handler (built with Microsoft's compiler) runs.

W4 — Internals panes and Windows labs

- **Build:** the debugged program's memory map (`VirtualQueryEx`), working set (`QueryWorkingSetEx`), heaps and thread stacks; a job object around it for limits and kill-on-close. Labs: SEH and termination handlers (Microsoft's compiler), a vectored exception handler, Windows Error Reporting and a minidump of your own crash, a threads lab (SRW locks, condition variables, the thread pool, one fiber), thread-local storage with `TlsAlloc`.
- **Done when:** region counts and types match VMMap for the same process; a minidump your code wrote opens in WinDbg at the faulting frame.

W5 — The engine on Windows

- **Build:** a service or background process owns the sessions; a named pipe whose security descriptor admits only your account; one I/O completion port as the event loop.
- **Done when:** reattach works as in L7, and Process Monitor shows another account's attempt to open the pipe failing with access denied.

W6 — Richter chapter 22, in the VM

- **Build:** the chapter's techniques, followed as the book teaches them, applied only to test programs your own debugger started, inside the VM.
- **Done when:** every `WriteFile` call your test program makes appears in a debugger pane.

W7 — Kernel slice and tracing, in the VM

- **Build:** a small driver of your own (a device object; create, close and device-control handlers; a kernel timer whose DPC counts ticks), built with the WDK or the Enterprise WDK, which means Microsoft's compiler; test signing on, Secure Boot off if the VM needs it; kernel debugging over the network. Then a real-time ETW consumer pane. Optional: a reader for the NTFS change journal, which is how *Windows Internals* chapter 11 gets any code at all.
- **Done when:** WinDbg's `!drvobj` shows your driver, a breakpoint in the DPC routine fires, and your ETW pane and Windows Performance Analyzer show the same process-start event.

Reading for this phase: Petzold 1–15 and 17 with W1–W2; Richter 1–26 across W1–W6; *Windows Internals* 3, 5, 8 with W3–W4, 7 and 10 with W5, 6, 8 and 10 with W7; Pavel Yosifovich, *Windows Kernel Programming*, for W7. Keep Microsoft Learn's pages on the pseudoconsole, the debugging API and the PE format open.

The network tab

The third tool is an intercepting proxy for your own applications, built on the engine's event loop, so a program you are debugging can have its traffic shown beside its breakpoints. It is the Burp-style target, folded into the same app.

N1 — HTTP parser and forward proxy

- **Build:** an HTTP/1.1 parser that works on byte buffers only (request line, headers, `Content-Length` and chunked bodies) and never trusts its input; fuzz it with the sanitizers on. The proxy runs inside the L7 engine's `epoll` loop. When the debugger starts a program, it sets `HTTP_PROXY` for it, so that program's requests land in the network tab.

- **Done when:** the fuzzer runs for an hour with no sanitizer report; a test client you wrote, started by the debugger, shows its requests in the network tab while you single-step it.
- **Read before:** TLPI 38 (untrusted input), 56–61 (sockets, server design). **Keep open:** RFC 9110 and RFC 9112.
- **Note:** a listener on 127.0.0.1 is reachable by every local user. On your own single-user machine that is acceptable; otherwise require a per-session token.

N2 — TLS interception, for your own test apps

- **Build:** a local certificate authority and per-host certificates, generated by `platform/tls.c`, the only file that includes OpenSSL. Trust that authority only in a test browser profile or test client, never system-wide.
- **Done when:** your test client's HTTPS requests appear decrypted in the tab, and the same client without your authority installed refuses the connection.
- **Keep open:** RFC 8446 (TLS 1.3); OpenSSL's documentation.

N3 — History store and replay

- **Build:** an append-only log file with an `mmap`ed index; a deliberate `fsync` policy; replay of any stored request.
- **Done when:** killing the engine with `kill -9` mid-capture loses at most the last record, and a replayed request gets the same response from your test server.

N4 — The network tab on Windows

- **Build:** Winsock with overlapped I/O on the W5 completion port.
- **Done when:** the N1–N3 tests pass on Windows.
- **Read before:** Petzold 23 (Winsock); Richter 10–11 (asynchronous I/O, the thread pool).

Graveyard of mistakes

Four mistakes this project will tempt you into, each compiled with the learn flags and run on Linux (the Windows part with MinGW) to show the failure.

1. Treating `long` as 64 bits, and a debuggee address as a pointer

On Linux `long` is 8 bytes (LP64). On 64-bit Windows it is 4 bytes (LLP64). C89 has no `long long` and no `uintptr_t`. An address inside the program you are debugging is a number in another process, so it is never a pointer in yours.

Naive (MinGW: `error: integer constant is too large for 'unsigned long' type`):

```
unsigned long addr = 0x7ffff7dd1000UL;   /* fine on Linux, an error on
Windows */
```

Veteran: `core/types.h` names the width once, and the build fails if it is wrong:

```
#if defined(_WIN64)
#  if defined(_MSC_VER)
typedef unsigned __int64 u64;
#  else
__extension__ typedef unsigned long long u64;  /* MinGW: C89 has no long long
*/
#  endif
#else
typedef unsigned long u64;                     /* LP64 Linux: long is 8 bytes
*/
#endif
/* C89 has no static_assert: an array of size -1 is a compile error. */
typedef char u64_is_8_bytes[sizeof(u64) == 8 ? 1 : -1];
#define U64(hi, lo) (((u64)(hi) << 32) | (u64)(lo))  /* no 64-bit literals */

u64 addr = U64(0x7fffU, 0xf7dd1000U);   /* builds on both */
```

2. One `wait` per SIGCHLD, and a child that inherits your signal mask

Standard signals are not queued. When 50 children exit together, the kernel sets one pending bit, and you get one signal. The engine also blocks SIGCHLD to read it with `signalfd`. A blocked mask survives `fork` and `exec`, so the shell in your terminal tab starts with SIGCHLD blocked, and its job control breaks quietly.

Naive (tested: `1 SIGCHLD read, 1 of 50 children reaped, 49 left as zombies`; the child's `/proc/self/status` shows `SigBlk: 0000000000010000`, which is bit 16, signal 17, SIGCHLD):

```
if (read(sfd, &si, sizeof si) == (ssize_t)sizeof si)
    waitpid(-1, &status, WNOHANG);          /* one signal, one child */
/* ...and in the child: */
execvp(argv[0], argv);                      /* runs with SIGCHLD still
blocked */
```

Veteran (tested: all 50 reaped; the child shows `SigBlk: 0000000000000000`):

```
while ((p = waitpid(-1, &status, WNOHANG)) > 0)   /* drain every dead child
*/
    if (p == child) alive = 0;
/* ...in the child, before exec: */
sigemptyset(&none);
sigprocmask(SIG_SETMASK, &none, NULL);      /* the engine's mask is not the
shell's */
execvp(argv[0], argv);
```

3. A plain flag as a hand-off between threads

C89 has no threads and no memory model. The compiler may load `ready` once and keep it in a register, and the CPU may make `line` visible after `ready`. `volatile` stops the first problem but not the second. POSIX defines `sem_post` and `sem_wait` as memory-synchronizing, so the data written before the post is visible after the wait.

Naive (tested: prints at `-O0`, hangs at `-O2` until `timeout` kills it with exit 124; `-fsanitize=thread` reports `data race flag.c:12`):

```
static int ready;
static void *reader(void *arg) { (void)arg; strcpy(line, "$ ls"); ready = 1;
return NULL; }
static void wait_line(void) { while (!ready) { } }
```

Veteran (tested at `-O2`: prints `render: $ ls`):

```
static sem_t full;                  /* sem_init(&full, 0, 0) in main */
static void *reader(void *arg) { (void)arg; strcpy(line, "$ ls");
sem_post(&full); return NULL; }
static void wait_line(void) { while (sem_wait(&full) != 0) { } }   /* retry
on EINTR */
```

4. A breakpoint written as a whole word

`PTRACE_POKETEXT` writes 8 bytes, not one. Writing `0xCC` as a word puts the trap in byte 0 and zeros the next 7 bytes of code. After the trap fires, `rip` points one byte past it, so continuing runs from the middle of an instruction.

Naive (tested: `breakpoint hit 1`, then `debuggee died: SIG11`):

```
ptrace(PTRACE_POKETEXT, pid, (void *)addr, (void *)0xCCL);   /* "write an
int3 there" */
```

Veteran (tested: three hits, then `debuggee exited 0 after 3 stops`):

```
struct bp { u64 addr; long saved; };

static int bp_set(pid_t pid, struct bp *b)
{
    long word;
    errno = 0;                    /* PEEKTEXT returns data: -1 can be a real
value */
    word = ptrace(PTRACE_PEEKTEXT, pid, (void *)b->addr, NULL);
    if (word == -1 && errno) return -1;
    b->saved = word;
    word = (long)(((unsigned long)word & ~0xFFUL) | 0xCCUL);   /* little-
endian: byte 0 */
    return (int)ptrace(PTRACE_POKETEXT, pid, (void *)b->addr, (void *)word);
}

static int bp_step_over(pid_t pid, struct bp *b)   /* after the SIGTRAP */
{
    struct user_regs_struct r;
    int st;
    ptrace(PTRACE_GETREGS, pid, NULL, &r);
    r.rip = b->addr;                                    /* back onto the real
instruction */
    ptrace(PTRACE_SETREGS, pid, NULL, &r);
    ptrace(PTRACE_POKETEXT, pid, (void *)b->addr, (void *)b->saved);
    ptrace(PTRACE_SINGLESTEP, pid, NULL, NULL);
    waitpid(pid, &st, 0);
    if (!WIFSTOPPED(st)) return -1;
    return bp_set(pid, b);                              /* armed for the next
hit */
}
```

The same steps hold on Windows. `WriteProcessMemory` writes only the bytes you give it, so the whole-word trap does not exist there, but the rewind does. You set `Rip` back by one in the `CONTEXT`, set the trap flag (`EFlags |= 0x100`) to step once, and re-arm on the `EXCEPTION_SINGLE_STEP`.

C89 at the edges

Strict C89 holds for every file you write; at the edges it meets headers and printf formats from later decades. Each row below was compiled here with your learn flags (GCC 13, MinGW for the Windows rows).

Edge	What happens under `-std=c89 -pedantic-errors`	What to do
Your own code	`//` comments, declarations after statements, designated initializers (`.a = f`) and `__VA_ARGS__` macros are all errors	`/* */`; declarations at the top of a block; Wayland listener structs filled in declaration order (`{ on_global, on_global_remove }`); a `log_msg(const char *fmt, ...)` function with `vfprintf` instead of a variadic macro
64-bit integers	No `long long` and no 64-bit literals; `long` is 4 bytes on Windows	`u64` from `core/types.h`, built with `U64(hi, lo)` (Graveyard 1)
Printing a `u64`	`%llu` warns ("ISO C90 does not support 'll'"); on MinGW `%lu` is the wrong width and `%I64u` warns	A 10-line `u64_hex(char out[17], u64 v)` in `core/`; it prints `00007ffff7dd1000` on both platforms
Feature-test macros	`pipe2`, `accept4` and other Linux-only calls are undeclared, so `implicit declaration` is an error	`#define _GNU_SOURCE` (or `_POSIX_C_SOURCE 200809L` for POSIX-only code) **before the first `#include`**. Defined after one, it does nothing. In a unity build that means once, at the top of `linux_main.c`; each lab sets its own

Edge	What happens under `-std=c89 -pedantic-errors`	What to do
Wayland headers	`wayland-util.h` uses `inline`	`-Dinline=__inline__`; your Emacs block already turns pkg-config's `-I` into `-isystem`, so the rest of the header is exempt
Linux debugger headers	`<elf.h>`, `<stdint.h>`, `<sys/ptrace.h>`, `<sys/epoll.h>`, `<sys/signalfd.h>`	Compile as they are, with `_POSIX_C_SOURCE 200809L`
OpenSSL 3	Its headers compile under strict C89 (3.0.13 here) because they are system headers	Keep them in `platform/tls.c` anyway, so the dependency stays in one file
Windows headers	`windows.h`, `winsock2.h` and `dbghelp.h` compile under MinGW with the same flags	MinGW is your strictness check for the Windows layer
Microsoft's compiler	`/Za` makes its C compiler strict C89, according to Microsoft's documentation, but it also turns Microsoft's keywords into plain identifiers, so don't expect files that include `windows.h` to build with it (not tested here: no MSVC). GCC and MinGW have no `__try` / `__except`; Microsoft's compiler and clang-cl do	`/Za` on `core/` as a second strictness check; Microsoft's compiler without it for the SEH labs and the W7 driver; MinGW for the rest of the Windows layer
Threads	C89 has no threads, atomics or memory model	POSIX mutexes and semaphores on Linux, SRW locks and events on Windows; never a shared flag (Graveyard 3)

Honest coverage

These counts are my own tally from this plan's milestone and lab lists, not a measured result. A chapter counts as covered when some milestone or lab makes you write code that depends on it. "Touched" means the code uses a small part of it. Reading a chapter

without writing code for it doesn't count.

Book	In the app	In a lab	Touched	Reading only	Not covered
The Linux Programming Interface (64 chapters)	36: 4–6, 12–13, 19–20, 22–34, 36–37, 41–42, 44, 49, 55–64	19: 7, 9, 14–17, 21, 38–40, 45–48, 50–54	3: 8 (users), 10 (time), 11 (limits)	4: 1–3, 43	2: 18 (directories, links), 35 (scheduling)
Windows via C/C++, 5th ed. (26)	17: 1–6, 9–10, 13–20, 22	8: 8, 11–12 (the W4 threads lab), 21, 23–26 (TLS, SEH, error reporting)	1: 7 (priorities)	—	—
Programming Windows, 5th ed. (23)	17: 1–15, 17, 23	—	2: 20–21 (threads and DLLs, done the Richter way)	—	4: 16 (palettes), 18 (metafiles), 19 (MDI), 22 (sound)
Windows Internals, 7th ed., Parts 1–2 (12)	7, through observation: 3–8, 10	—	1, optional: 11 (change journal)	2: 1–2	2: 9 (virtualization), 12 (startup)

Windows Internals is descriptive, so "covered" there means you built something whose behaviour that chapter explains and checked it with WinDbg, Process Monitor or an ETW trace.

The two TLPI gaps are cheap to close if you want them: a recursive directory walker with `nftw` and hard and symbolic links (18), and a lab that pins a busy loop with `sched_setaffinity` and changes its `nice` value while `perf stat` watches (35).

Reading map

Read each block before the milestone that needs it, and keep the reference column open while you code. You learn C, the C runtime and assembly along the way, in that order, because the debugger keeps asking you about each one.

Phase	Read before	Keep open	The question it answers
Foundation, before L1	Kernighan & Ritchie, *The C Programming Language*, 2nd ed., the whole book with its exercises; *Computer Systems: A Programmer's Perspective* (CS:APP), 3rd ed., chapters 2–3	Your Third Document (the flags and what each one catches)	What does my C turn into in memory and in instructions?
The C runtime, the warm-ups	CS:APP 7 (linking) and 8 (exceptional control flow); LWN's *How programs get run: ELF binaries* (2015); TLPI 6 (process memory layout)	`readelf`, `objdump -d`, `elf(5)`	What runs before `main`, and how does a file on disk become a running process?
L1–L3	The TLPI chapters listed in each milestone; Drew DeVault, *The Wayland Protocol*	The DEC ANSI parser page, XTerm's control sequences, `pty(7)`, `termios(3)`	What is a terminal, really?
L4–L6	Eli Bendersky, *How debuggers work*, parts 1–3 (short, free); Sy Brand, *Building a Debugger* (No Starch, 2025); TLPI 12 (`/proc`) with L4 and 28 (`clone`) with L6	`ptrace(2)`, the DWARF 5 standard, the x86-64 System V ABI, Intel's manual (volume 2 for instructions, volume 3 for debug registers)	How does one process stop, read and change another?
L7	The listed chapters, plus TLPI 19 (`inotify`) and 23 (`timerfd`)	`epoll(7)`, `unix(7)`	How does a long-running service stay correct?

Phase	Read before	Keep open	The question it answers
N1–N4	TLPI 58–61; W. Richard Stevens, *UNIX Network Programming*, vol. 1, if you want more depth	RFC 9110, RFC 9112, RFC 8446, OpenSSL's documentation	What is actually on the wire?
W1–W7	As listed under the Windows milestones	Microsoft Learn: the debugging functions, the PE format, the pseudoconsole, the x64 calling convention	The same questions, asked of Windows
Labs	Each lab's TLPI chapter; CS:APP 9.9 for the allocator	The man page of every call the lab makes	—

How much assembly

You need to read x86-64, not write it. CS:APP chapter 3 is enough to read `objdump -d` and to know what `call`, `ret` and `push` do to `rsp`. `C-c m a` shows the `-O2` assembly of your own code with each source line beside it, so practise there first. Look up other instructions in Intel's volume 2 as you meet them. Windows uses a different calling convention: the first four arguments go in `rcx`, `rdx`, `r8` and `r9`, and the caller reserves 32 bytes of shadow space. Learn it at W3, when your stack walker first sees it.

Day 1 rules, goodies and bloat, feedback loop

Five rules from the first commit

1. **Every function is `static`, and each platform is one translation unit.** The compiler then sees the whole program: it can inline across files, and `-Wunused-function` reports dead code the moment it appears.
2. **`core/` includes only the C89 standard library.** L1's `grep` test enforces it. That is what makes the Windows port a new platform layer instead of a rewrite.

3. **Every byte from outside is hostile.** That means ELF, DWARF and PE files, the memory of a crashed program, and HTTP bytes. Parsers take a pointer and a length, check the bounds before every read, and are fuzzed with the sanitizers on. A debugger that crashes on bad debug info fails exactly when you need it.
4. **Memory is grouped by lifetime: one arena per frame, per debug session and per request.** Freeing one arena frees everything with that lifetime, so there is no per-object `free` to forget and no use-after-free between lifetimes.
5. **Every milestone's done-test is written in the README and run against a reference tool**: gdb, `readelf`, `strace`, `ps`, WinDbg, VMMap. Your mental model is only checked when a tool that is known to be right agrees with you.

Goodies to use from day 1

- **Linux:** one `epoll` loop where everything is a file descriptor (`signalfd`, `timerfd`, `inotify`, sockets, the PTY master); `O_CLOEXEC` on every descriptor; `mmap` for ELF and DWARF, so the file itself is the data structure; `pread` on `/proc/PID/mem`, which reads a page in one call where `PTRACE_PEEKTEXT` needs 512.
- **Windows:** one I/O completion port as the loop; SRW locks; `VirtualQueryEx`; minidumps.
- **Both:** arenas; `u64` and fixed-size cells; positional struct initializers; your own `u64_hex`.

Bloat to ban

- **Build systems** (CMake, autotools): the unity build and `build.sh` already do the job.
- **GUI toolkits** (GTK, Qt) and Windows class libraries (MFC, ATL): Wayland's `wl_shm` and plain Win32 are the lesson.
- **libelf, libdwarf, libunwind:** they do exactly the parts you are building the debugger to learn. Use `readelf` and `objdump` to check your output instead.
- **HTTP and config-parsing libraries:** you write the HTTP parser; configuration is a C header you recompile.
- **Signal handlers in the app:** `signalfd` only; handlers stay in the L3 lab.
- **`_GNU_SOURCE`, C99 or C++ in `core/`.**

The feedback loop

Question	Tool
Did I write undefined behaviour?	`C-c C-c` (sanitizers), `C-c m u` (`-O0` against `-O2`), `C-c m c` (analyzer)
What does my debugger actually ask the kernel?	`strace -e trace=ptrace,wait4 ./workbench`, **without `-f`**. With `-f`, strace traces the child first, so its `PTRACE_TRACEME` fails with `EPERM` and your debugger hangs (both tested).
Did my breakpoint write the right byte?	The same strace line shows it: `PEEKTEXT ... [0xe5894855fa1e0ff3]` then `POKETEXT ... 0xe5894855fa1e0fcc`. Only the low byte changed, from `f3` (the first byte of `endbr64`, which Ubuntu's GCC puts at the start of every function) to `cc`.
Do I agree with gdb?	The same debuggee in gdb: `info registers`, `stepi`; `readelf --debug-dump=decodedline` against your line table
What is on the wire?	`curl -x http://127.0.0.1:PORT`, `openssl s_client`, `ss -tnp`
Where does the time go?	`C-c m p` (`perf stat`); `perf record -g` (frame pointers are on)
Is this struct laid out well?	`C-c m l` (`pahole`)
What is the compositor doing? Which libraries loaded?	`WAYLAND_DEBUG=1`; `LD_DEBUG=libs`
Windows	WinDbg (`!address`, `~*k`), Process Monitor, VMMap, Windows Performance Recorder and Analyzer

Sources and testing

What "tested" means here: compiled with your learn flags under GCC 13.3 on Ubuntu 24.04 and run there; Windows code compiled with MinGW-w64 GCC under the same flags but not run, because there is no Windows machine in this environment. Anything about Microsoft's compiler comes from its documentation.

- POSIX.1-2024, Base Definitions, §4.15.2 *Memory Synchronization* (the list includes `sem_post` and `sem_wait`): pubs.opengroup.org
- Microsoft, */Za, /Ze (Disable Language Extensions)*: learn.microsoft.com
- LWN, *How programs get run: ELF binaries* (2015): lwn.net
- Eli Bendersky, *How debuggers work, part 1* (2011): eli.thegreenplace.net
- Sy Brand, *Building a Debugger* (No Starch Press): O'Reilly listing
- Pavel Yosifovich, *Windows Kernel Programming*, 2nd ed.: leanpub.com
- Paul Williams, *A parser for DEC's ANSI-compatible video terminals*: vt100.net